Empathy Map

SAYS

I need a tool that doesn't just find bugs, but guarantees their absence (Soundness).

Python's dynamic nature makes it hard to track variable types statically.

Are there too many false positives in this report?

I want to catch security gaps early, before they reach production.

THINKS

Is it possible to achieve 100% soundness without making the analysis too slow?

I hope this tool supports the latest Python libraries we are using.

Manual code reviews are taking too long; I need to automate this 'health check'.

If the analysis is too complex, will other developers actually use it?

who?

Static Analysis Developer/User

DOES

Integrates static analysis into the CI/CD pipeline to vet every commit.

Maps source code to Abstract Syntax Trees (AST) or Control Flow Graphs (CFG).

Researches Abstract Interpretation and formal mathematical models.

Traces data flow from user inputs to critical API calls (Taint Analysis).

FEELS

Frustrated by "silent" runtime errors like null pointer dereferences.

Anxious about missing a critical security vulnerability (e.g., SQL Injection).

Relieved when a tool provides a clear fix for a complex logic bug.

Overwhelmed by long reports containing thousands of warnings.

PAIN

High computational overhead of deep static analysis.

False positives requiring human judgment and time.

Difficulty analyzing "dynamic" Python features (like getattr or eval).

Complexity of maintaining a "sound" analyzer as the language evolves.

GAIN

Drastic reduction in debugging time and maintenance effort.

Guaranteed code quality and adherence to safety standards.

"Shift-left" security: finding bugs while code is still being written.

Higher confidence in the reliability of safety-critical systems.

Problem Statements:

1. The dynamic nature of high-level languages like Python—specifically features such as dynamic typing, late binding, and reflection—renders traditional static analysis imprecise and prone to false negatives.

2. Current analysis tools for Python often sacrifice soundness (the guarantee that no bugs are missed) for speed, leaving safety-critical applications vulnerable to "silent" runtime errors and security exploits. Furthermore, analyzing Python at the source-code level often fails to capture the actual execution logic of the interpreter.